

课程笔记

Survey of LLM Agent Reasoning & Planning, PDDL

五道口纳什 & Claude

2026 年 3 月 28 日

Agent Planning

Reasoning models ...

Prompt-based: Plan-and-Solve, ...

Search-based: ToT, MCTS...

Test-time scaling: DeepConf

Neural-Symbolic: DSL/PDDL ...

视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 约 32 分钟

视频链接: <https://www.bilibili.com/video/BV1wjY5zyEki>

目录

1 系列概述与课程导引	2
1.1 系列将覆盖的主题	2
1.2 本章小结	2
2 Reasoning: 语言模型智能的基石	2
2.1 什么是 Reasoning	2
2.2 Reasoning 与 Agent 能力的关系	3
2.3 两类经典推理方式	3
2.4 Reasoning Model: 隐式的 Long Reasoning	3
2.5 本章小结	3
3 Planning: LLM Agent 的核心挑战	3
3.1 什么是 Planning	3
3.2 Planning 方法分类 Survey	4
3.2.1 分解 (Decomposition)	4
3.2.2 选择 (Selection)	4
3.2.3 外部求解器 (External Solver)	4
3.2.4 反思 (Reflection & Critique)	4
3.3 Planning 方法的个人分类	5
3.4 本章小结	5
4 经典案例深度解析	5
4.1 Alpha Geometry: Neuro-Symbolic 方案	5
4.1.1 Alpha Geometry 第一代	5
4.1.2 Alpha Geometry 第二代	5
4.2 DeepConf: 基于置信度的 Majority Voting	6
4.3 UCLA IMO 方案: Self-Critique Workflow	6
4.4 本章小结	6
5 从 ReAct 到 Plan-and-Solve	6
5.1 ReAct 的局限性	6
5.2 Plan-and-Solve Prompting	6
5.3 Dynamic Plan-and-Solve	7
5.4 本章小结	7
6 PDDL: Neuro-Symbolic Planning	7
6.1 语言模型直接规划的困境	7
6.2 LM+P: 扬长避短的方案	8
6.3 本章小结	8
7 总结与延伸	8
7.1 核心观点回顾	8
7.2 延伸思考	9

1 系列概述与课程导引

本期是 Modern Agents 付费系列的第一期，从宏观视角串讲了整个系列未来将涉及的核心主题，并以 **Reasoning & Planning** 作为切入点，对大语言模型（LLM）在规划领域的能力与不足进行了系统性的 survey。

1.1 系列将覆盖的主题

整个 Modern Agents 系列计划覆盖以下几大板块：

1. **Reasoning and Planning**：语言模型的推理与规划能力，这是所有后续内容的基石。
2. **经典 Workflow**：包括 Self-Critique（生成-评估-迭代循环）、Multi-Agent System（多智能体协作，核心在于 Separation of Concern）。
3. **Context Engineering**：对语言模型相关技术的整体打包，重点关注 Memory 设计。
4. **Self-Evolving**：以 AlphaEvolve、GPA 等为代表的自我进化技术。
5. **框架实践**：LangChain / LangGraph 等框架的使用。
6. **Graph-based 方法**：GraphRAG 等 Neuro-Symbolic 方案。
7. **工具设计与封装**：为语言模型设计合适的 Tool / Server 以补足其短板。
8. **Test-Time Computing**：并行思考（DeepConf）、Majority Voting 等推理时计算技术。

1.2 本章小结

本系列的核心理念是：在不断的实践中掌握更多行之有效的 know-how，通过 Agent 的设计和开发来最大化语言模型的表现。

2 Reasoning：语言模型智能的基石

2.1 什么是 Reasoning

Reasoning 的核心定义

Reasoning（推理）是一种“无中生有”的能力——从已知且有限的前提（Premise）出发，沿着推理链推导出更多、更丰富、更有深度的结论。

经典的演绎推理三段论就是一个典型的例子：

- 大前提：所有人都会死
- 小前提：苏格拉底是人
- 结论：苏格拉底也会死

模型真正的智能就是通过 Reasoning 来体现的。当我们对模型的回答感到 impressed 或 disappointed 时，本质上都是对其 Reasoning 能力的一种感知——超出预期令人惊叹，低于预期则令人失望。

2.2 Reasoning 与 Agent 能力的关系

Reasoning 是一切高级能力的基础

如果模型没有 Reasoning 的能力：

- 就没有 **Planning**：因为 Planning 需要分解任务、Think Ahead、想多步
- 就没有 **Reflection / Critique**：因为反思需要从已有前提产生大量新结论
- 就没有有效的 **Agentic** 行为：因为 Agent 需要基于推理来决策下一步动作

2.3 两类经典推理方式

1. **演绎推理 (Deduction)**：前向推理，是确定性推理。从大前提和小前提出发推导出必然结论。
2. **归纳推理 (Induction)**：是或然性推理。从有限的观察中归纳出一般性结论，前提越多结论越自信，但不能保证绝对正确（如”天鹅都是白色的”）。

2.4 Reasoning Model：隐式的 Long Reasoning

当前的 Reasoning Model（如 DeepSeek R1、OpenAI O 系列）追求的是一种**隐式的 Long Reasoning**：

- 复杂问题 → 模型自发产生长的思考过程
- 简单问题 → 产生短的思考过程
- 这是在训练阶段让模型自发学会的能力

2.5 本章小结

Reasoning 是语言模型一切高级能力的基石。无论是 Planning、Reflection 还是 Agent 行为，都依赖于模型从有限前提中推导出新结论的能力。现代 Reasoning Model 正在通过 RL 训练来增强这种隐式的长链推理能力。

3 Planning：LLM Agent 的核心挑战

3.1 什么是 Planning

Planning 的核心含义

Planning 是对任务的分解——不管是隐式还是显式的，都要：

1. 识别用户的目标
2. 分解任务为可执行的子步骤
3. Think Ahead：提前想多步
4. Think Multiple Stages：考虑多个阶段

引入 Planning 的原因在于：语言模型在解决复杂的、长周期的、Multi-Hop 的问题时存在明显缺陷。通过显式或隐式的 Planning 机制，可以 Leverage 模型能力，最大化其在复杂任务上的 Performance。

3.2 Planning 方法分类 Survey

根据 2024 年初的一篇经典 Survey（关于 LLM 做 Planning 的综述），可以将方法分为以下几大类：

3.2.1 分解 (Decomposition)

- **CoT (Chain of Thought)**: “Let’s think step by step”，让模型一步步求解问题，内部涉及对问题的自由分解。
- **ReAct**: 不断和环境交互——Thought → Action → Observation 循环，在交互过程中完成任务分解。
- **Plan-and-Solve**: 显式地形成一个 Plan，然后沿着 Plan 去 Solve。
- **PoT (Program of Thought)**: 将思考过程转化为可执行的程序。

3.2.2 选择 (Selection)

- **ToT (Tree of Thoughts)**: 用树结构管理搜索过程，从一个节点出发扩展多个候选方案，基于评分选择最有潜力的节点进一步探索。
- **MCTS (Monte Carlo Tree Search)**: 基于 UCB 值综合考虑探索和利用，选择节点进行拓展。关键在于 **Simulation**（对节点进行打分）和 **Selection**（选择最优节点）。

状态爆炸问题

在 ToT 和 MCTS 中，从每个节点出发语言模型可以产生 multiple 的 proposal，这会导致指数级的状态爆炸。因此必须引入有效的评分机制 (Score)，对节点做剪枝 (Prune)，只对有潜力的节点进一步探索。

3.2.3 外部求解器 (External Solver)

如 PDDL Planner，用于 Robotics 等场景中的 Planning 任务。语言模型直接产生 Plan 时往往不 work，引入外部符号求解器可以大幅提升可靠性。

3.2.4 反思 (Reflection & Critique)

通过 Self-Refine 机制，模型对自身输出进行反思和改进，在记忆 (Memory) 层面也可以引入 Reflection 来形成更高层次的结论。

Generative Agents 中的 Memory Reflection

在 Generative Agents 中，如果只记录浅层事实（“他在读论文”），价值有限。通过触发 Reflection——综合多次观察，可以形成更高层结论（“这是一个对科研非常有热情的人”），这才是有深度的记忆。

3.3 Planning 方法的个人分类

作者按自己的维度将 Planning 方法分为五类：

类别	代表方法
Reasoning Model（隐式）	DeepSeek R1, OpenAI O 系列
Plan-based（显式）	ReAct, Plan-and-Solve
Search-based	ToT, LLM + MCTS
Test-Time Scaling	Majority Voting, DeepConf
Neuro-Symbolic	LM+P, Alpha Geometry, PDDL

表 1: Planning 方法分类

3.4 本章小结

LLM 在面对复杂长周期问题时，直接生成 Plan 容易出错。可以通过分解（CoT/ReAct）、搜索（ToT/MCTS）、外部求解器（PDDL）、反思（Self-Critique）等多种策略来提升 Planning 质量。

4 经典案例深度解析

4.1 Alpha Geometry: Neuro-Symbolic 方案

4.1.1 Alpha Geometry 第一代

第一代 Alpha Geometry 的核心思路：

1. 将原始几何问题翻译成**符号语言**
2. 让**符号引擎**尝试直接求解
3. 如果无法求解，交给**语言模型**提出构造（如添加辅助线）
4. 符号引擎基于新构造推导出新的 facts
5. 循环直到形成完整的解决方案

例如：已知 $AB = AC$ （等腰三角形），要证明两个角相等。仅凭 $AB = AC$ 符号引擎无法直接推导；但语言模型建议在 BC 中点加一个 D 点并连接 AD ，就可以通过相似三角形、对角相等性质完成证明。

4.1.2 Alpha Geometry 第二代

第二代引入了**多路并行搜索**：

- 不止一个语言模型提 proposal，而是 multiple 的 neural model 并行搜索
- 各路搜索中间产生的新 facts 是**共享的**，可以形成新的前提（states），再交给不同 model 继续探索
- 这种并行思考策略启发了后续 Gemini Deep Think 版本的设计

4.2 DeepConf: 基于置信度的 Majority Voting

DeepConf 的核心创新:

- 引入 **Token Confidence** 概念
- 对每个 token 计算置信度, 再 aggregate 得到整个 trace (完整解决方案) 的置信度
- 基于置信度加权做 **Majority Voting**, 而非简单的多数表决

4.3 UCLA IMO 方案: Self-Critique Workflow

Generate-Verify-Correct 循环

UCLA 求解 IMO 的方案是一个典型的 Self-Critique workflow:

1. Step 1-2: 产生一个初始 proposal (solution)
2. Step 3: 交给 Critique 做 verification, 给出具体的问题和意见
3. Step 4: 语言模型基于反馈做 correction
4. 循环执行, 直到连续 5 次通过验证 (接受) 或连续 10 次失败 (拒绝)

其关键词是 **capable**——模型本身有能力解决问题, 只是需要合适的 workflow 来激发。

4.4 本章小结

三个经典案例分别展示了 Neuro-Symbolic (Alpha Geometry)、Test-Time Scaling (DeepConf) 和 Self-Critique (UCLA IMO) 三种 Planning 策略, 它们殊途同归——通过精心设计的 workflow 来最大化模型的推理潜力。

5 从 ReAct 到 Plan-and-Solve

5.1 ReAct 的局限性

ReAct 通过 Thought $\rightarrow$ Action $\rightarrow$ Observation 循环来处理复杂问题, 但存在一个关键问题:

Context Issues

当 ReAct 循环特别长时, 会出现严重的上下文问题:

- 语言模型会 **Lost in the Middle**——迷失在冗长的上下文中
- 忘记用户的原始任务是什么
- 忘记自己做过哪些事情、得到了什么结果
- 无法产生下一步合理的 Action

5.2 Plan-and-Solve Prompting

Plan-and-Solve 是对经典 Zero-shot CoT 的改进。其 Prompting 方式为:

“Let’s first understand the problem and devise a step-by-step plan to solve it. Then let’s carry out the plan and solve the problem step by step.”

对比经典的 “Let’s think step by step”，Plan-and-Solve 显式地在 Prompting 中引入了 Plan 的概念，在复杂问题上大约有几个百分点的提升。

5.3 Dynamic Plan-and-Solve

社区对 Plan-and-Solve 做了进一步改进，引入了**动态性**：

刚中之脑 vs. 真实交互

Plan 是在头脑中形成的（刚中之脑），没有和环境真实交互。当真实执行时会产生新的 feedback，这时需要 **Re-Plan**——将新的反馈引入到规划机制中进行动态修正。

Dynamic Plan-and-Solve 的完整流程：

1. **Plan**：Planner 自顶向下形成一个 plan（List of Steps / To-Do List）
2. **Execute**：每个 step 交给一个 ReAct Agent 去执行
3. **Re-Plan**：执行后基于新的 feedback，决定是直接结束还是重新规划

Re-Plan 的输入包括：原始问题 + 原始 Plan + 过去执行的步骤及其结果。

经济性考量：强弱模型组合

从经济角度，可以用**强模型**做 Planner 和 Re-Plan（因为规划决定了整体质量上限），用**弱模型**做具体的 ReAct 执行（因为具体执行步骤相对简单）。这样在成本和效果之间取得平衡。

5.4 本章小结

从 ReAct 到 Plan-and-Solve 再到 Dynamic Plan-and-Solve，体现了一条清晰的演进线：从纯反应式交互，到显式引入规划，再到动态调整规划以弥合“刚中之脑”与真实交互之间的 gap。

6 PDDL: Neuro-Symbolic Planning

6.1 语言模型直接规划的困境

在 Blocked World 等经典 Planning 问题中（如积木世界），语言模型直接产生 Plan 时很容易出错。例如 GPT-4 在第四步就产生了错误——因为涉及状态约束（机械臂一次只能拿一个 block、block 必须在桌面或某一个位置上），语言模型很容易在内部产生明显的逻辑错误。

6.2 LM+P: 扬长避短的方案

LM+P 的核心思想

- **语言模型不擅长的**: 执行精确的 Reasoning 或解决 Planning 问题
- **语言模型擅长的**: 翻译——将自然语言转换为符号语言
- **PDDL Solver 擅长的**: 基于完整的 Domain 定义和 Goal State, 产生可靠的执行序列

因此, 让语言模型做翻译, 让 PDDL Solver 做规划求解。

具体流程:

1. 有一个 Domain 的 PDDL 定义 (描述环境中有哪些 action、precondition、effect)
2. 用户给出自然语言指令 (如“把苹果放在冰箱里”)
3. 语言模型**翻译**为 Goal State (如 apple_1 在 fridge_1 中)
4. PDDL Solver 基于 Domain 定义和 Goal State, **求解**出具体的 Plan
5. 语言模型再把每个步骤**翻译**回自然语言

语言模型不应被寄予过高期望

即使是强如 Gemini 2.5 Pro, 在解方程 $5.9 = x + 5.11$ 这样简单的问题时也会犯错。我们不应该预期语言模型可以 solve 所有问题——这时一个简单的计算器 tool 就够了。要做系统级的设计, 在合适的环节使用合适的工具。

6.3 本章小结

PDDL 代表了 Neuro-Symbolic 方案的精髓: 让语言模型做它擅长的翻译, 让专业的符号求解器做精确的规划。这种扬长避短的设计思路是构建可靠 Agent 的关键。

7 总结与延伸

7.1 核心观点回顾

1. **显式或隐式的 Plan 是必要且有效的**: 对于语言模型求解复杂的长周期问题, Plan 机制可以显著提升表现。
2. **Top-Down 与 Bottom-Up 的结合**: Top-Down 做任务分解形成 Plan, Bottom-Up 通过真实交互的 feedback 进行 Re-Plan, 弥合头脑规划与真实执行之间的 Gap。
3. **感知模型的能力与特点, 扬长避短**: 设计合适的 Workflow 和 Tool 来激发模型能力, 规避其短板。
4. **系统级的设计**: 语言模型 + Tools + Prompts 需要统筹考虑, 形成稳定高效的解决方案。

7.2 延伸思考

- **Workflow 设计的重要性**: UCLA 的 IMO 方案证明, 模型本身有能力 (capable) 解决问题, 关键在于 workflow 的设计是否能最大化地激发模型潜力。
- **从 Agentic Workflow 到 Autonomous Agent**: 目前大量 Coding Agent (Claude Code、Codex、Cursor) 都采用预定义的 workflow 而非完全自主的 Agent, 因为可控、可靠、可预测。完全自主的 Agent 是终极目标, 但受限于当前模型能力。
- **Agent 的设计和开发可能与模型训练同等重要**: 从实用角度来看, 通过精心设计 Agent 来达到更好的 performance, 可能比训练更强的模型更为经济高效。